



Problémamegoldás programozással — 2. gyakorlat

A mai óra célja: megérteni, hogyan tudunk egy utasítássorozatot ismételten végrehajtani, hogyan vezérelhetjük az ismétlések számát vagy feltételét, hogyan generálhatunk véletlen számokat, és hogyan használhatjuk a C# beépített matematikai függvényeit.

A mai óra felépítése

1. Kérdések az előző laborhoz
2. Miért van szükség ciklusokra?
3. Elöl tesztelő és hátul tesztelő ciklusok
4. `for` ciklus
5. `while` ciklus
6. `do-while` ciklus
7. Végtelen ciklusok
8. Véletlenszám-generálás
9. Matematikai függvények — `System.Math`
10. Rövid ellenőrző kérdések
11. Gyakorlati feladatmegoldás

? Kérdések az előző laborhoz

Mielőtt továbblépünk, nézzük meg, maradt-e kérdés az előző gyakorlat anyagából.

Az előző óra legfontosabb elemei

- utasítások és pontosvessző,
- kommentek,
- `Console.Write()` és `Console.WriteLine()` ,
- változók és típusok,
- deklaráció és értékadás,
- `Console.ReadLine()` ,
- típuskonverzió,
- `if / else if / else` ,
- aritmetikai, összehasonlító és logikai operátorok.



Miért van szükség ciklusokra?

Tegyük fel, hogy ki szeretnénk írni a számokat 0-tól 9-ig.

Megtehetnénk így:

```
Console.WriteLine(0);
Console.WriteLine(1);
Console.WriteLine(2);
Console.WriteLine(3);
Console.WriteLine(4);
Console.WriteLine(5);
Console.WriteLine(6);
Console.WriteLine(7);
Console.WriteLine(8);
Console.WriteLine(9);
```

Ez működik, de:

- sok ismétlődő kódot írunk,
- nehéz módosítani,
- nagyobb ismétlésszámnál teljesen életszerűtlen,
- könnyű hibázni.

Ugyanez ciklussal

```
for (int i = 0; i < 10; i++)
{
    Console.WriteLine(i);
}
```

✅ A ciklus lehetővé teszi, hogy ugyanazt az utasítássorozatot többször végrehajtsuk anélkül, hogy újra és újra le kellene írunk.



A ciklus alapötlete

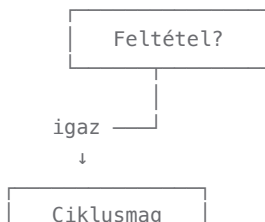
Egy ciklusnak általában két fontos része van:

1. Ciklusfeltétel

Meghatározza, hogy **folytatódjon-e még az ismétlés**.

2. Ciklusmag

Az az utasítás vagy utasítássorozat, amelyet újra és újra végrehajtunk.





Elöl tesztelő és hátul tesztelő ciklusok

A legfontosabb különbség az, hogy **mikor vizsgáljuk meg a ciklusfeltételt**.

Elöl tesztelő ciklus

A feltételt a ciklusmag **előtt** vizsgáljuk meg.

C#-ban ilyen:

- while
- for

Ha a feltétel már a ciklus indulásakor hamis, a ciklusmag **egyszer sem fut le**.

Hátul tesztelő ciklus

A feltételt a ciklusmag **után** vizsgáljuk meg.

C#-ban ilyen:

- do-while

A ciklusmag legalább egyszer biztosan végrehajtódik.

Elöl vagy hátul tesztelő?

Ciklus	Feltételvizsgálat	Minimum lefutás
for	ciklusmag előtt	0
while	ciklusmag előtt	0
do-while	ciklusmag után	1

Gondolkodjunk!

Melyiket választanánk?

„Addig kérjük be a jelszót, amíg helyes nem lesz.”

Itt a felhasználótól **legalább egyszer biztosan kérnünk kell egy jelszót**.

Ezért a `do-while` természetes választás lehet.

A for ciklus

A `for` ciklust akkor célszerű használni, amikor **előre ismert vagy jól meghatározható, hányszor szeretnénk végrehajtani a ciklusmagot**.

Alapforma

```
for (int i = 0; i < 10; i++)  
{  
    // ciklusmag  
}
```

A `for` utáni zárójelben található a ciklus vezérlése.

```
for ( inicializátor ; feltétel ; iterátor )
```

A for ciklus három része

```
for (int i = 0; i < 10; i++)  
{  
    Console.WriteLine(i);  
}
```

<code>int i = 0</code>	<code>; i < 10</code>	<code>; i++</code>
1	2	3

1. Inicializátor

```
int i = 0;
```

- létrehozza a ciklusváltozót,
- kezdőértéket ad neki,
- **csak egyszer fut le**,
- még a ciklus indulása előtt.

2. Feltétel

```
i < 10
```

Amíg a feltétel igaz, a ciklus folytatódik.

A feltétel a ciklusmag **minden lefutása előtt** kiértékelődik.

3. Iterátor

```
i++
```

A ciklusmag **minden lefutása után** végrehajtódik.

Gyakran a ciklusváltozó növelésére vagy csökkentésére használjuk.

```
i++;
```

ugyanazt jelenti, mint:

```
i = i + 1;
```



A for ciklus végrehajtási sorrendje

```
for (int i = 0; i < 3; i++)  
{  
    Console.WriteLine(i);  
}
```

Végrehajtás

1. `int i = 0`
2. `i < 3 ?` → igaz
3. ciklusmag → kiírja: 0
4. `i++`
5. `i < 3 ?` → igaz

```
6. ciklusmag → kiírja: 1
7. i++
8. i < 3 ? → igaz
9. ciklusmag → kiírja: 2
10. i++
11. i < 3 ? → hamis
12. ciklus vége
```

Eredmény

```
0
1
2
```

⚠ A `i < 3` miatt a `3` már nem kerül kiírásra.



Hányszor fut le?

```
for (int i = 0; i < 10; i++)
{
    Console.WriteLine(i);
}
```

A lehetséges `i` értékek:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Tehát a ciklusmag **10 alkalommal** fut le.



Visszafelé számlálás

```
for (int i = 10; i > 0; i--)
{
    Console.WriteLine(i);
}
```

Eredmény:

```
10
9
8
```

7
6
5
4
3
2
1

Az iterátor most:

```
i--
```

ami ugyanaz, mint:

```
i = i - 1;
```

Nem kötelező egyesével lépni

```
for (int i = 0; i <= 20; i = i + 2)
{
    Console.WriteLine(i);
}
```

Eredmény:

0
2
4
6
8
10
12
14
16
18
20

 Az iterátor tetszőleges olyan művelet lehet, amely a ciklusváltozót a kilépési feltétel felé mozdítja.

Végtelen for ciklus

```
for (int i = 10; i > 0; i++)
{
    // ciklusmag
}
```

Mi történik?

- `i` kezdetben `10` ,
- a feltétel: `i > 0` ,
- minden iteráció után `i` **növekszik**,
- így egyre távolabb kerülünk attól, hogy `i <= 0` legyen.

✗ A feltétel soha nem válik hamissá → **végtelen ciklus**.



A while ciklus

A `while` ciklust akkor használjuk, amikor az ismétlések száma **nem feltétlenül ismert előre**, de meg tudunk adni egy feltételt, amely alapján a ciklus folytatódjon.

Szintaxis

```
while (feltétel)
{
    // ciklusmag
}
```



Egyszerű while példa

```
int i = 0;

while (i < 10)
{
    Console.WriteLine(i);
    i = i + 1;
}
```

A működés ugyanaz, mint ennél:

```
for (int i = 0; i < 10; i++)
{
    Console.WriteLine(i);
}
```

A `while` esetén nekünk kell külön megoldani:

1. a kezdőértéket,
2. a feltételt,
3. a változó módosítását.



Mikor jó választás a `while` ?

Például:

„Addig kérjünk be adatot, amíg a felhasználó nem írja be azt, hogy `stop` .”

Nem tudjuk előre, hány alkalommal fog próbálkozni.



Ilyenkor egy feltételes ciklus, például `while` , jó megoldás lehet.



`while` ciklusnál figyeljünk a változásra

```
int i = 0;

while (i < 10)
{
    Console.WriteLine(i);
}
```

Az `i` értéke soha nem változik, ezért:

```
i = 0
i < 10 → mindig igaz
```

A ciklus végtelen lesz.

Javítás

```
int i = 0;

while (i < 10)
{
    Console.WriteLine(i);
    i++;
}
```

💡 Mindig gondoljuk végig: **mi fogja egyszer hamissá tenni a ciklusfeltételt?**

A do-while ciklus

A do-while hátul tesztelő ciklus.

Szintaxis

```
do
{
    // ciklusmag
}
while (feltetel);
```

⚠️ A while (feltetel) után itt **pontosvessző van**.

★ A do-while legfontosabb tulajdonsága

A ciklusmag **legalább egyszer biztosan lefut**.

1. ciklusmag
2. feltételvizsgálat
3. ha igaz → újra ciklusmag
4. ha hamis → ciklus vége

Példa — bemenet stop -ig

```
string input;

do
{
    input = Console.ReadLine();
}
while (input != "stop");
```

A változó:

```
string input;
```

kezdetben még nem kap értéket.

Ez itt nem probléma, mert:

1. először lefut a ciklusmag,
2. ott értéket kap,
3. és csak ezután vizsgáljuk meg a feltételt.

✓ A `do-while` különösen praktikus, amikor **először mindenképpen be kell kérnünk valamit**, és csak utána tudjuk eldönteni, folytassuk-e.

while vagy do-while ?

while

```
while (feltetel)
{
    // ciklusmag
}
```

Először megvizsgálja a feltételt.

Ha az hamis, egyszer sem fut le.

do-while

```
do
{
    // ciklusmag
}
while (feltetel);
```

Először végrehajtja a ciklusmagot, és csak utána vizsgálja meg a feltételt.

∞ Végtelen ciklusok

Végtelen ciklusról beszélünk, ha a ciklusfeltétel **soha nem válik hamissá**.

Tipikus véletlen hiba

```
int i = 0;

while (i < 10)
{
    Console.WriteLine(i);
}
```

Tudatos végtelen ciklus

```
while (true)
{
    // ciklusmag
}
```

A `true` mindig igaz, ezért a ciklus önmagától soha nem ér véget.

⚠ A végtelen ciklus lehet **hiba**, de bizonyos programokban **tudatos tervezési döntés** is.



Ciklusválasztási segédlet

Ha...	Jó kiindulás
Ismerjük az ismétlések számát	<code>for</code>
Nem tudjuk, hányszor kell ismételni	<code>while</code>
Legalább egyszer biztosan végre kell hajtani	<code>do-while</code>
A program folyamatosan fusson	tudatos végtelen ciklus

📌 Ez nem merev szabály. Sok feladat több ciklustípussal is megoldható — a cél a **legegyszerűbb és legérthetőbb megoldás** kiválasztása.



Véletlenszám-generálás

C#-ban véletlenszerűnek tűnő értékek előállításához használhatjuk a `Random` típust.

Például:

- kockadobás,
- játékok,
- szimulációk,
- tesztadatok,

- véletlen események.

Álvéletlen szám

A számítógép által előállított „véletlen” számok tipikusan **álvéletlen számok**.

Ez azt jelenti, hogy a számok véletlenszerű mintázatot mutatnak, de valójában egy determinisztikus algoritmus állítja elő őket.

Generátor létrehozása

```
Random generator = new Random();
```

Ezután a `generator` objektumot használhatjuk új véletlen értékek előállítására.

`Random.Next()`

Egész véletlen szám

```
int x = generator.Next();
```

Felső határ megadása

```
int y = generator.Next(13);
```

Lehetséges eredmények:

0 ... 12

 A felső határ **nem része** a tartománynak.

Alsó és felső határ

```
int z = generator.Next(69, 420);
```

Lehetséges eredmények:

69 ... 419

Tehát:

$$\text{alsó határ} \leq \text{eredmény} < \text{felső határ}$$


Példa — kockadobás

```
Random generator = new Random();  
  
int dobas = generator.Next(1, 7);  
  
Console.WriteLine(dobas);
```

A lehetséges eredmények:

1, 2, 3, 4, 5, 6

Miért 7 a felső határ?

Mert a felső határ kizáró.



Random.NextDouble()

```
double y = generator.NextDouble();
```

Az eredmény:

$$0.0 \leq y < 1.0$$

Vagyis:

- a 0 előfordulhat,
- az 1 már nem.



Véletlen lebegőpontos szám más tartományban

Ha 0 és 10 közötti értéket szeretnénk:

```
double x = generator.NextDouble() * 10;
```

Általánosan egy [min, max) tartományhoz:

```
double x = min + generator.NextDouble() * (max - min);
```

💡 A mai feladatok között is lesz olyan, ahol a véletlenszám-generálást ciklussal kell kombinálni.



Matematikai függvények — System.Math

A C# számos matematikai műveletet készen biztosít számunkra.

Ezek a Math osztályon keresztül érhetők el.

```
Math.Abs(...)  
Math.Pow(...)  
Math.Sqrt(...)  
Math.Log10(...)  
Math.Sin(...)  
Math.Cos(...)  
Math.Tan(...)
```

+ Abszolútérték — Math.Abs()

```
double x1 = Math.Abs(-1.23);
```

Eredmény:

```
1.23
```

Egész számmal is működik:

```
int x2 = Math.Abs(-1);
```

Eredmény:

Hatványozás — `Math.Pow()`

```
double y1 = Math.Pow(1.2, 3.4);
```

Jelentése:

$$1.2^{3.4}$$

Általánosan:

```
Math.Pow(alap, kitevo)
```

√ Négyzetgyök — `Math.Sqrt()`

```
double y2 = Math.Sqrt(123.4);
```

Kiszámítja:

$$\sqrt{123.4}$$

Egyszerű példa:

```
double x = Math.Sqrt(25);
```

Eredmény:

Tízes alapú logaritmus

```
double y3 = Math.Log10(42);
```

Kettes alapú logaritmus

```
double x = Math.Log2(8);
```

Természetes logaritmus

```
double y = Math.Log(10);
```

Trigonometriai függvények

C#-ban használhatunk például:

```
Math.Sin(...)  
Math.Cos(...)  
Math.Tan(...)
```

Példa

```
double z1 = Math.Cos(0);
```

Fok vagy radián?

A trigonometriai függvények paraméterét **radiánban** kell megadni.

Nem fokban.

Példa — 90°

$90^\circ = \pi / 2$ radián

C#-ban:

```
double z2 = Math.Sin(Math.PI / 2);
```

Az eredmény:

1

π és e

A fontos matematikai konstansok szintén elérhetők a `Math` osztályon keresztül.

```
Math.PI  
Math.E
```

⚠ Ezek **nem metódusok**, ezért nincs utánuk `()` .

Helyes:

```
double x = Math.PI;
```

Nem helyes:

```
double x = Math.PI();
```

Függvényhívások értelmezése

```
double eredmeny = Math.Sqrt(25);
```

Itt:

- `Math` → az osztály,
- `Sqrt` → a használt matematikai függvény / metódus,
- `25` → paraméter,
- `5` → visszatérési érték,
- az eredményt az `eredmeny` változóban tároljuk.

Ez ugyanazt az általános mintát követi, amit már korábban láttunk:

```
Console.WriteLine("Hello");
```

és:

```
int.Parse("123");
```



Kombináljuk az eddigieket!

```
Random generator = new Random();

for (int i = 0; i < 5; i++)
{
    int x = generator.Next(1, 7);

    if (x == 6)
    {
        Console.WriteLine("Hatost dobtunk!");
    }
    else
    {
        Console.WriteLine(x);
    }
}
```

Ebben egyszerre használunk:

- változót,
- Random objektumot,
- for ciklust,
- értékadást,
- if / else elágazást,
- összehasonlító operátort,
- konzolos kiírást.



A programozásban az új feladatok egyre inkább a már ismert elemek **összekapcsolásáról** szólnak.



Tipikus hibák ciklusoknál

1. A ciklusváltozó nem változik

```
int i = 0;
```

```
while (i < 10)
{
    Console.WriteLine(i);
}
```

✗ Végtelen ciklus.

2. Rossz irányba módosítjuk

```
for (int i = 10; i > 0; i++)
{
    Console.WriteLine(i);
}
```

✗ `i` nő, miközben a kilépéshez csökkennie kellene.

3. Hibás végfeltétel

```
for (int i = 0; i <= 10; i++)
```

Ez `0 ... 10`, tehát **11 iteráció**.

Míg:

```
for (int i = 0; i < 10; i++)
```

ez `0 ... 9`, tehát **10 iteráció**.

4. Hiányzó pontosvessző `do-while` után

Helyes:

```
do
{
    // ...
}
while (feltétel);
```



Gyors gondolkodó kérdések

1. Mi a ciklusmag?
2. Mit jelent az, hogy egy ciklus elől tesztelő?

3. Mi a legfontosabb különbség a `while` és a `do-while` között?
4. Mikor célszerű `for` ciklust használni?
5. A `for (int i = 0; i < 10; i++)` hány alkalommal fut le?
6. Mi történik, ha egy `while` ciklus feltétele soha nem lesz hamis?
7. Mit adhat vissza a `generator.Next(1, 7)`?
8. Benne lehet-e a `7` az előző eredmények között?
9. Milyen tartományból ad értéket a `NextDouble()`?
10. Milyen mértékegységben várja a `Math.Sin()` a szöveget?
11. Mi a különbség a `Math.PI` és a `Math.Sqrt(...)` használata között?



Problémamegoldási minta ciklusos feladatokhoz

Mielőtt kódolunk, tegyük fel a következő kérdéseket.

1. Mit kell ismételni?

Mi legyen a **ciklusmag**?

2. Hányszor kell ismételni?

Ha tudjuk:

valószínűleg `for`.

Ha nem tudjuk:

valószínűleg feltételes ciklus.

3. Mi dönti el, hogy folytassuk-e?

Mi a **ciklusfeltétel**?

4. Mi változik minden iterációban?

Például:

`i` növekszik
próbálkozások száma nő
új véletlen szám keletkezik
felhasználói bemenet változik

5. Mi fogja biztosan befejezni a ciklust?

Ha erre nincs jó válaszunk, könnyen végtelen ciklust készítünk.



Példa — gondolkodjunk kódolás előtt

Feladat:

Írjuk ki 1 -től N -ig a számokat.

Bemenet

N

Mit ismétlünk?

egy szám kiírását

Hányszor?

N alkalommal

Mi változik?

i

Jó ciklusválasztás

for

Csak ezután írjuk meg a kódot.



Másik példa — jelszó bekérése

Feladat:

Addig kérjük be a jelszót, amíg helyes nem lesz.

Tudjuk előre a próbálkozások számát?

Nem.

Legalább egyszer kérnünk kell jelszót?

Igen.

Természetes választás

do-while

💡 Előbb modellezzük a problémát, utána választunk szintaxist.

✅ Mit szeretnék, ha az óra végére magabiztosan felismernének?

- ☐ Mi a ciklus és mi a ciklusmag?
- ☐ Mi a ciklusfeltétel?
- ☐ Mi a különbség az elől és hátul tesztelő ciklus között?
- ☐ Mikor célszerű `for` ciklust használni?
- ☐ Mit jelent az inicializátor, a feltétel és az iterátor?
- ☐ Hogyan működik a `while` ciklus?
- ☐ Miért fut le a `do-while` legalább egyszer?
- ☐ Hogyan keletkezik végtelen ciklus?
- ☐ Hogyan hozunk létre `Random` generátort?
- ☐ Mit jelent, hogy a `Next()` felső határa kizáró?
- ☐ Milyen tartományból ad értéket a `NextDouble()` ?
- ☐ Hogyan használjuk a `Math.Abs()` , `Math.Pow()` és `Math.Sqrt()` függvényeket?
- ☐ Miért radiánban dolgoznak a trigonometriai függvények?
- ☐ Mi a `Math.PI` ?
- ☐ Hogyan tervezzünk meg egy ciklusos feladatot kódolás előtt?